

21 % 6; /* 结果是 3 */21 / 6; /* 结果是 3 */

21 % 7; /* 结果是 0 */21 / 7; /* 结果是 3 */

-21 % -8; /* 结果是 -5 */-21 / -8; /* 结果是 2 */

21 % -5; /* 结果是 1 */21 / -5; /* 结果是 -4 */

142

4.2 节练习

练习 4.4: 在下面的表达式中添加括号, 说明其求值的过程及最终结果。编写程序编译该 (不加括号的) 表达式并输出其结果验证之前的推断。

12 / 3 * 4 + 5 * 15 + 24 % 4 / 2

练习 4.5: 写出下列表达式的求值结果。

(a) -30 * 3 + 21 / 5 (b) -30 + 3 * 21 / 5

(c) 30 / 3 * 21 % 5 (d) -30 / 3 * 21 % 4

练习 4.6: 写出一条表达式用于确定一个整数是奇数还是偶数。

练习 4.7: 溢出是何含义? 写出三条将导致溢出的表达式。

4.3 逻辑和关系运算符

关系运算符作用于算术类型或指针类型, 逻辑运算符作用于任意能转换成布尔值的类型。逻辑运算符和关系运算符的返回值都是布尔类型。值为 0 的运算对象 (算术类型或指针类型) 表示假, 否则表示真。对于这两类运算符来说, 运算对象和求值结果都是右值。

表 4.2: 逻辑运算符和关系运算符			
结合律	运算符	功能	用法
右	!	逻辑非	!expr
左	<	小于	expr < expr
左	<=	小于等于	expr <= expr
左	>	大于	expr > expr
左	>=	大于等于	expr >= expr
左	==	相等	expr == expr
左	!=	不相等	expr != expr
左	&&	逻辑与	expr && expr
左		逻辑或	expr expr

逻辑与和逻辑或运算符

对于逻辑与运算符 (&&) 来说, 当且仅当两个运算对象都为真时结果为真; 对于逻辑或运算符 (||) 来说, 只要两个运算对象中的一个为真结果就为真。

逻辑与运算符和逻辑或运算符都是先求左侧运算对象的值再求右侧运算对象的值, 当且仅当左侧运算对象无法确定表达式的结果时才会计算右侧运算对象的值。这种策略称为短路求值 (short-circuit evaluation)。

- 对于逻辑与运算符来说, 当且仅当左侧运算对象为真时才对右侧运算对象求值。
- 对于逻辑或运算符来说, 当且仅当左侧运算对象为假时才对右侧运算对象求值。

第3章中的几个程序用到了逻辑与运算符，它们的左侧运算对象是为了确保右侧运算对象求值过程的正确性和安全性。例如85页的循环条件：

```
index != s.size() && !isspace(s[index])
```

首先检查 `index` 是否到达 `string` 对象的末尾，以此确保只有当 `index` 在合理范围之内时才会计算右侧运算对象的值。

举一个使用逻辑或运算符的例子，假定有一个存储着若干 `string` 对象的 `vector` 对象，要求输出 `string` 对象的内容并且在遇到空字符串或者以句号结束的字符串时进行换行。使用基于范围的 `for` 循环（参见3.2.3节，第81页）处理 `string` 对象中的每个元素：

```
// s 是对常量的引用；元素既没有被拷贝也不会被改变
for (const auto &s : text) {      // 对于 text 的每个元素
    cout << s;                    // 输出当前元素
    // 遇到空字符串或者以句号结束的字符串进行换行
    if (s.empty() || s[s.size() - 1] == '.')
        cout << endl;
    else
        cout << " "; // 否则用空格隔开
}
```

输出当前元素后检查是否需要换行。`if` 语句的条件部分首先检查 `s` 是否是一个空 `string`，如果是，则不论右侧运算对象的值如何都应该换行。只有当 `string` 对象非空时才需要第二个运算对象的值，也就是检查 `string` 对象是否是以句号结束的。在这条表达式中，利用逻辑或运算符的短路求值策略确保只有当 `s` 非空时才会用下标运算符去访问它。

◀ 143

值得注意的是，`s` 被声明成了对常量的引用（参见2.5.2节，第61页）。因为 `text` 的元素是 `string` 对象，可能非常大，所以将 `s` 声明成引用类型可以避免对元素的拷贝；又因为不需要对 `string` 对象做写操作，所以 `s` 被声明成对常量的引用。

逻辑非运算符

逻辑非运算符（`!`）将运算对象的值取反后返回，之前我们曾经在3.2.2节（第79页）使用过这个运算符。下面再举一个例子，假设 `vec` 是一个整数类型的 `vector` 对象，可以使用逻辑非运算符将 `empty` 函数的返回值取反从而检查 `vec` 是否含有元素：

```
// 输出 vec 的首元素（如果有的话）
if (!vec.empty())
    cout << vec[0];
```

子表达式

```
!vec.empty()
```

当 `empty` 函数返回假时结果为真。

关系运算符

顾名思义，关系运算符比较运算对象的大小关系并返回布尔值。关系运算符都满足左结合律。

因为关系运算符的求值结果是布尔值，所以将几个关系运算符连写在一起会产生意想不到的结果：

```
// 哎哟! 这个条件居然拿 i < j 的布尔值结果和 k 比较!
if (i < j < k) // 若 k 大于 1 则为真!
```

if 语句的条件部分首先把 i、j 和第一个<运算符组合在一起, 其返回的布尔值再作为第二个<运算符的左侧运算对象。也就是说, k 比较的对象是第一次比较得到的那个或真或假的结果! 要想实现我们的目的, 其实应该使用下面的表达式:

```
// 正确: 当 i 小于 j 并且 j 小于 k 时条件为真
if (i < j && j < k) { /* ...*/ }
```

相等性测试与布尔字面值

如果想测试一个算术对象或指针对象的真值, 最直接的方法就是将其作为 if 语句的条件:

```
if (val) { /* ...*/ }           // 如果 val 是任意的非 0 值, 条件为真
if (!val) { /* ...*/ }         // 如果 val 是 0, 条件为真
```

144 在上面的两个条件中, 编译器都将 val 转换成布尔值。如果 val 非 0 则第一个条件为真, 如果 val 的值为 0 则第二个条件为真。

有时会试图将上面的真值测试写成如下形式:

```
if (val == true) { /* ...*/ } // 只有当 val 等于 1 时条件才为真!
```

但是这种写法存在两个问题: 首先, 与之前的代码相比, 上面这种写法较长而且不太直接(尽管大家都认为缩写的形式对初学者来说有点难理解); 更重要的一点是, 如果 val 不是布尔值, 这样的比较就失去了原来的意义。

如果 val 不是布尔值, 那么进行比较之前会首先把 true 转换成 val 的类型。也就是说, 如果 val 不是布尔值, 则代码可以改写成如下形式:

```
if (val == 1) { /* ...*/ }
```

正如我们已经非常熟悉的那样, 当布尔值转换成其他算术类型时, false 转换成 0 而 true 转换成 1 (参见 2.1.2 节, 第 32 页)。如果真想知道 val 的值是否是 1, 应该直接写出 1 这个数值来, 而不要与 true 比较。



进行比较运算时除非比较的对象是布尔类型, 否则不要使用布尔字面值 true 和 false 作为运算对象。

4.3 节练习

练习 4.8: 说明在逻辑与、逻辑或及相等性运算符中运算对象求值的顺序。

练习 4.9: 解释在下面的 if 语句中条件部分的判断过程。

```
const char *cp = "Hello World";
if (cp && *cp)
```

练习 4.10: 为 while 循环写一个条件, 使其从标准输入中读取整数, 遇到 42 时停止。

练习 4.11: 书写一条表达式用于测试 4 个值 a、b、c、d 的关系, 确保 a 大于 b、b 大于 c、c 大于 d。

练习 4.12: 假设 i、j 和 k 是三个整数, 说明表达式 i!=j<k 的含义。

4.4 赋值运算符

赋值运算符的左侧运算对象必须是一个可修改的左值。如果给定

145

```
int i = 0, j = 0, k = 0;      // 初始化而非赋值
const int ci = i;            // 初始化而非赋值
```

则下面的赋值语句都是非法的：

```
1024 = k;                    // 错误：字面值是右值
i + j = k;                   // 错误：算术表达式是右值
ci = k;                      // 错误：ci 是常量（不可修改的）左值
```

赋值运算的结果是它的左侧运算对象，并且是一个左值。相应的，结果的类型就是左侧运算对象的类型。如果赋值运算符的左右两个运算对象类型不同，则右侧运算对象将转换成左侧运算对象的类型：

```
k = 0;                       // 结果：类型是 int，值是 0
k = 3.14159;                 // 结果：类型是 int，值是 3
```

C++11 新标准允许使用花括号括起来的初始值列表（参见 2.2.1 节，第 39 页）作为赋值语句的右侧运算对象：

C++
11

```
k = {3.14};                  // 错误：窄化转换
vector<int> vi;               // 初始为空
vi = {0,1,2,3,4,5,6,7,8,9}; // vi 现在含有 10 个元素了，值从 0 到 9
```

如果左侧运算对象是内置类型，那么初始值列表最多只能包含一个值，而且该值即使转换的话其所占空间也不应该大于目标类型的空间（参见 2.2.1 节，第 39 页）。

对于类类型来说，赋值运算的细节由类本身决定。对于 vector 来说，vector 模板重载了赋值运算符并且可以接收初始值列表，当赋值发生时用右侧运算对象的元素替换左侧运算对象的元素。

无论左侧运算对象的类型是什么，初始值列表都可以为空。此时，编译器创建一个值初始化（参见 3.3.1 节，第 88 页）的临时量并将其赋给左侧运算对象。

赋值运算满足右结合律

赋值运算符满足右结合律，这一点与其他二元运算符不太一样：

```
int ival, jval;
ival = jval = 0;             // 正确：都被赋值为 0
```

因为赋值运算符满足右结合律，所以靠右的赋值运算 `jval=0` 作为靠左的赋值运算符的右侧运算对象。又因为赋值运算返回的是其左侧运算对象，所以靠右的赋值运算的结果（即 `jval`）被赋给了 `ival`。

对于多重赋值语句中的每一个对象，它的类型或者与右边对象的类型相同、或者可由右边对象的类型转换得到（参见 4.11 节，第 141 页）：

```
int ival, *pval;             // ival 的类型是 int; pval 是指向 int 的指针
ival = pval = 0;             // 错误：不能把指针的值赋给 int
string s1, s2;
s1 = s2 = "OK";              // 字符串字面值"OK"转换成 string 对象
```

因为 `ival` 和 `pval` 的类型不同，而且 `pval` 的类型（`int*`）无法转换成 `ival` 的类型